Introduction
Project Outline
Exploring The Data (EDA)
Model Set-Up
Fitting The Models
Model Results
Putting the Best Models to Test
Conclusion

Code

Predicting Virality of Urban Dictionary Terms

Sarah Feuer

UCSB PSTAT131 W24 Final

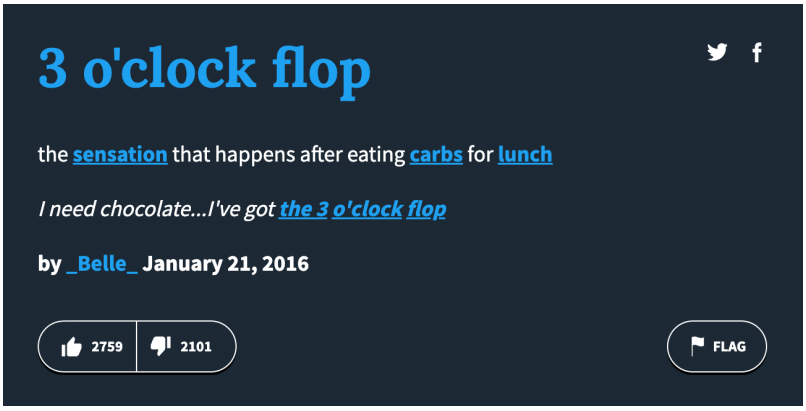
Introduction

My family and friends will be the first to tell you that I shorten every word I possibly can.
Syllables quickly get sliced off my sentences and the people close to me quickly get dubbed with a spunky nickname. Why not? it saves time and adds spice to any mundane conversation – or shall I say dane convo. The avid authors of Urban Dictionary are cut from the same cloth, and I admire their witty word play. Urban Dictionary is an online crowdsourced dictionary that catalogs slang terms and cultural references, providing user-generated definitions that often capture informal, humorous, and niche phrases used mainly in online subcultures. Not to mention, Urban Dictionary words can - and have - broken into the mainstream, changing a whole generation's cultural conversation. What could be seen as silly Urban Dictionary terms, can transform into powerful words used to talk about political campaigns, social movements, and funny memes - how cool! Not to mention, Urban Dictionary is kinda silly, and it's fun to do some real data exploration and machine learning techniques on a goofy topic, becuase why not. So, what makes a "successful" Urban Dictionary word that appeals to the masses - that's the journey we'll embark on together.

.

Project Outline

The aim of this exploration is to predict our responde variable - the number of upvotes - an Urban Dictionary would recieve based on categories outlined below. To build our final regression model, we will first tidy our data and get a feel for our data set using exploratory data analysis (EDA). Moving on to the more technical component in model building, we will split our data into a training/testing set, design a recipe, and set folds for 10-fold cross validation. Then, our 5 models will be used with our training set, to determine the most accurate. Whichever model performs the strongest, will then be used to fit to our testing data set and analyze how effective our model is. All these steps will be outlined in more detail as we go along, so let's get to it!



Exploring The Data (EDA)

Loading and Exploring the Data

The data set that will serve as the building block for this exploration is The Kaggle dataset, "Urban Dictionary Terms" by author ATHONTZ which can be found here (https://www.kaggle.com/datasets/athontz/urban-dictionary-terms/data). Let's take a look at the raw data we're working with - and tidy from there! Here's one entry for reference.

	word	definition	author	tags	up	down	date
2	trump basket	A toilet bowl.	jknightx	[]	729	634	March 23, 2017

Hide

```
dim(urban_dictionary_data)
```

```
## [1] 4272    7
```

Codebook

The data set consists of 4272 observations and 7 variables:

`word` - the slang term added to Urban Dictionary, a character variable.
`definition` - the definition of the slang term, a character variable.
`author` - the user account who posted the term, a character variable.
`tags` - a list of the hashtags used, a character variable.
`up` - number of upvotes the term got, a numeric variable.
`down` - number of downvotes the term got, a numeric varaible.
`date` - the date the term was added to Urban Dictionary, a character variable.

As the aim of this project is to predict virality, upvotes seem like the obvious metric to be our response variable and measure of sucess. Looking ahead towards model construction – 1. number of tags, 2. whether the `word` / `defintion` / `tags` include political terms, and 3. whether the `word` / `defintion` / `tags` include explicit terms and topics, will serve as insightful predictors. Number of tags will be a helpful predictor because hashtags may publicize the `word` on the website, getting it on more pages, and therefore reaching more eyes. Whether the entry includes political or explicit terms will be a good indication on virality because when looking through Urban Dictionary entries, it's clear the vast majority of terms relate to one or the other. So, it's worthwhile to check if upvotes correlate to either of the categories. Let's work on constructing the data frame to meet our response/predictor needs.

First, let's drop the columns irrelevant to our data exploration, `author`, `down`, `date`.

Hide

```
drop <- c("author", "down", "date")
urban_dictionary_data = urban_dictionary_data[,!(names(urban_dictionary_data) %in%
drop)]
```

Describing the Predictors

I'll start the initial construction of the political and explicit predictors, by sourcing lists of common words and phrases that fall into each category. (sourced from Glossary of Political Terms (https://www.englishclub.com/glossaries/political-terms.php) and Ultimate Curse Word Bracket (https://deadspin.com/behold-the-ultimate-curse-word-bracket-457043269))

The political list has 180 words, and the explicit list 65. For the sake of model integrity, these lists obviously don't cover every political and explicit word, but for the sake of this exploration, they should be sufficient in categorizing whether entries are related to political or explicit topics, because out of 4272 entries, even if some political/explicit indicators are false positives or false negatives, the main trends should still break through.

Let's add a `political` and `explicit` column to out data frame if any of the words in the two lists are in the `word` or `defintion`

Hide

```
# add a column if the term is political or not
urban_dictionary_data <- urban_dictionary_data %>%
  mutate(
    word = tolower(word),  # Convert 'word' column to lowercase
    definition = tolower(definition),  # Convert 'definition' column to lowercase
    political = ifelse(grepl(paste(politics_list, collapse = "|"), word) | grepl(pa
ste(politics_list, collapse = "|"), definition), TRUE, FALSE)
  )

# add a column if the term is explicit or not
urban_dictionary_data <- urban_dictionary_data %>%
  mutate(
    word = tolower(word),  # Convert 'word' column to lowercase
    definition = tolower(definition),  # Convert 'definition' column to lowercase
    explicit = ifelse(grepl(paste(explicit_list, collapse = "|"), word) | grepl(pas
te(explicit_list, collapse = "|"), definition), TRUE, FALSE)
  )
```

As a sanity check, let's see what proportion of `words` are categorized at political and/or explicit.

Hide

```
total_true_political <- sum(urban_dictionary_data$political)
total_true_political
```

```
## [1] 475
```

Hide

```
total_true_explicit <- sum(urban_dictionary_data$explicit)
total_true_explicit
```

```
## [1] 1178
```

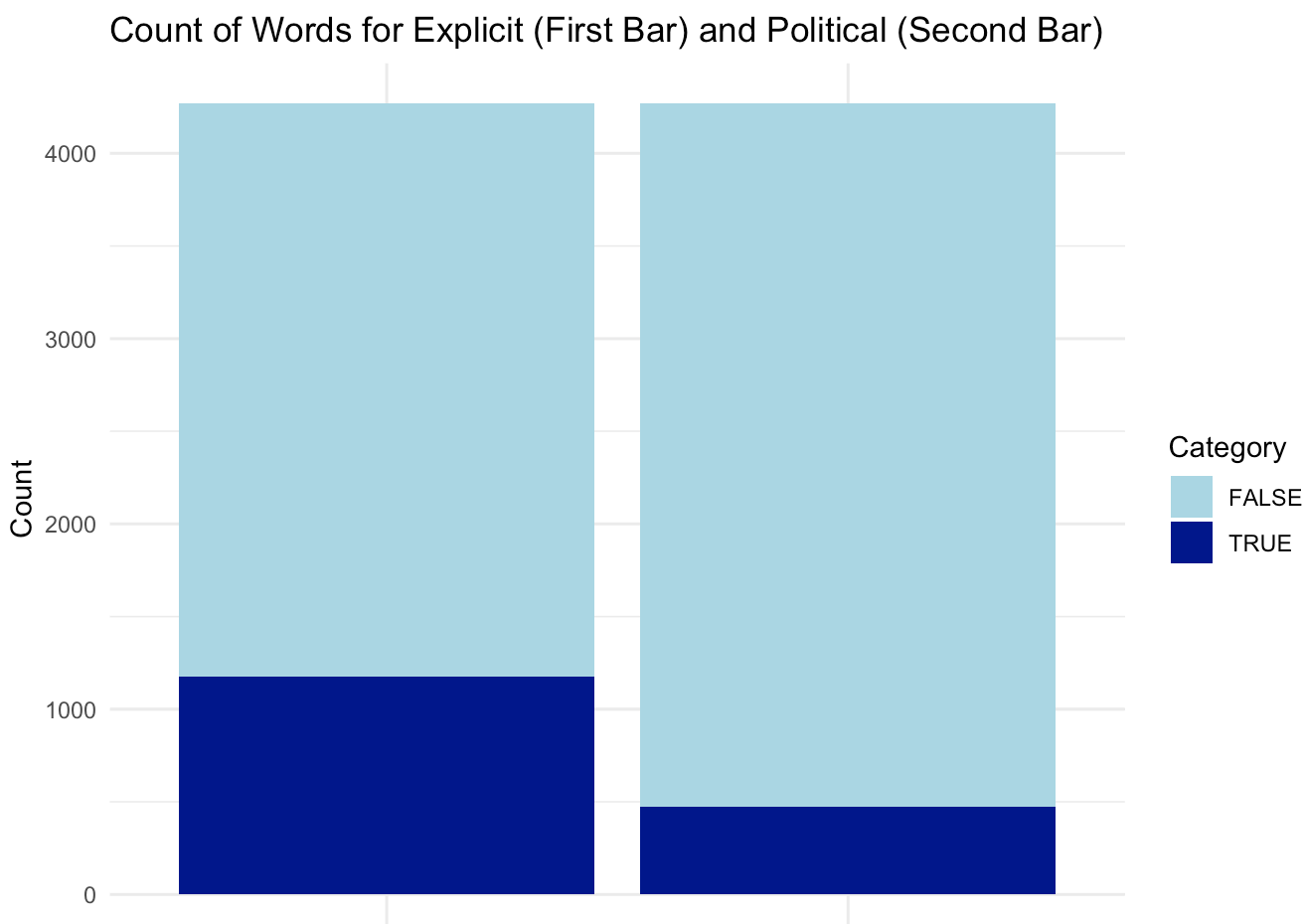
Hide

```
counts_political <- table(urban_dictionary_data$political)
counts_explicit <- table(urban_dictionary_data$explicit)

# Create data frames for plotting
plot_data_political <- data.frame(Category = names(counts_political), Count = as.nu
meric(counts_political), Type = "Political")
plot_data_explicit <- data.frame(Category = names(counts_explicit), Count = as.nume
ric(counts_explicit), Type = "Explicit")

# Combine the data frames
combined_plot_data <- rbind(plot_data_political, plot_data_explicit)

# Plot
ggplot(combined_plot_data, aes(x = Type, y = Count, fill = Category)) +
  geom_bar(stat = "identity") +
  labs(title = "Count of Words for Explicit (First Bar) and Political (Second Ba
r)",
       x = NULL, y = "Count") +
  scale_fill_manual(values = c("FALSE" = "#ADD8E6", "TRUE" = "#00008B")) +
  theme_minimal() +
  theme(axis.text.x = element_blank())
```



Finally, let's add in a column, `has_tags` to indicate whether the `word` was posted with tags or not

Hide

```
# add a column if the term has hashtags or not
urban_dictionary_data <- urban_dictionary_data %>%
  mutate(has_tags = ifelse(tags == "[]", FALSE, TRUE))

urban_dictionary_data <- urban_dictionary_data[, c("word", "definition", setdiff(na
mes(urban_dictionary_data), c("word", "definition")))]
```

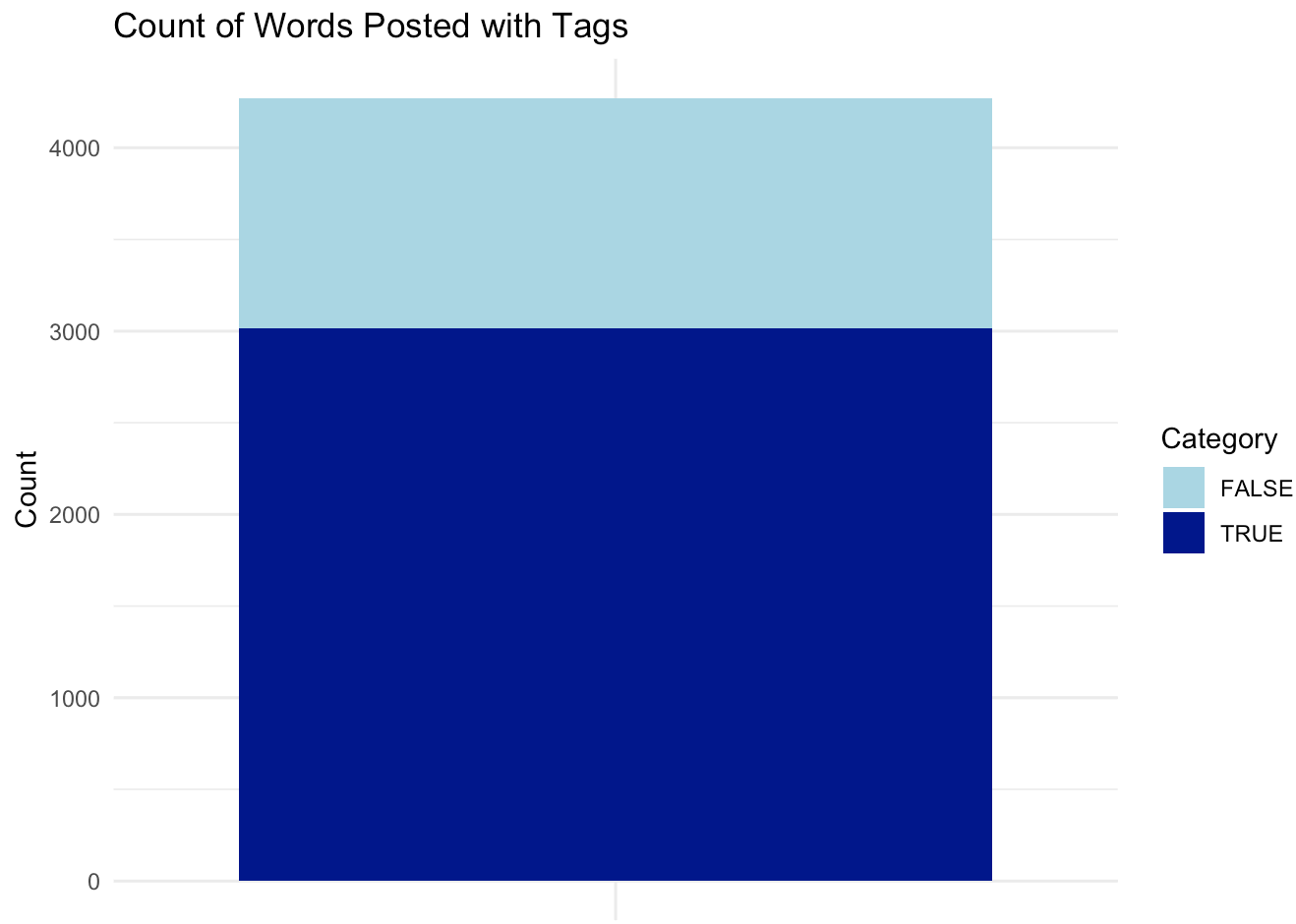
As a final sanity check, let's see what proportion of words are posted with tags.

Hide

```
counts_tags <- table(urban_dictionary_data$has_tags)

# Create data frame for plotting
plot_data_tags <- data.frame(Category = names(counts_tags), Count = as.numeric(coun
ts_tags), Type = "Tags")

# Plot
ggplot(plot_data_tags, aes(x = Type, y = Count, fill = Category)) +
  geom_bar(stat = "identity") +
  labs(title = "Count of Words Posted with Tags",
       x = NULL, y = "Count") +
  scale_fill_manual(values = c("FALSE" = "#ADD8E6", "TRUE" = "#00008B")) +
  theme_minimal() +
  theme(axis.text.x = element_blank())
```



A little over 3000 - nearly 75% of our data - has tags!

Hide

```
# Print the updated dataframe
kable(urban_dictionary_data[2:3, ])
```

	word	definition	tags	up	political	explicit	has_tags
2	trump basket	a toilet bowl.	[]	729	TRUE	FALSE	FALSE
3	wealth care	excellent health care reserved exclusively for the wealthy.	[u'#insurance', u'#rich', u'#america', u'#capitalism', u'#socialism']	1072	TRUE	FALSE	TRUE

Voila! We now have a workable data set that we will use to fit our model. As a recap, our variables are now:

`word` - the slang term added to Urban Dictionary, a character variable.
`definition` - the definition of the slang term, a character variable.
`tags` - a list of the hashtags used, a character variable.
`up` - number of upvotes the term got, a numeric variable.
`political` - indicator variable if the term is related to political topics, a logical variable.
`explicit` - indicator variable if the term is related to explicit topics, a logical variable.
`has_tags` - indicator variable if the term was posted with tags, a logical variable.

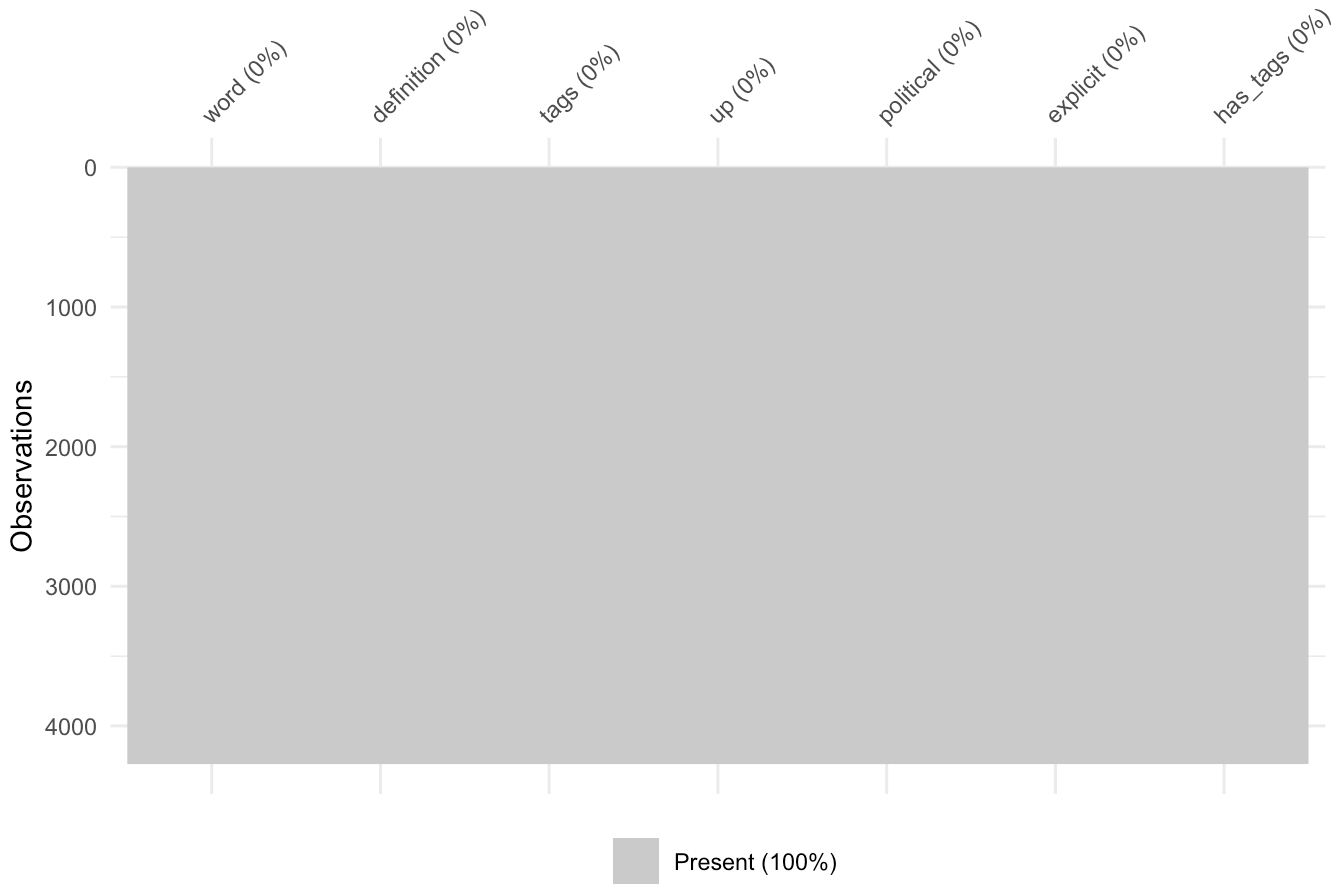
Visual EDA

Missing Data Plot

Even though it's unlikely that there are missing values because the data is just scraped from Urban Dictionary, it's important to check any data set for missingness that would obstruct predictions later on, or decrease the accuracy of our model.

Hide

```
vis_miss(urban_dictionary_data)
```



The plot indicates that there are no missing values across all our variables. Phew! - that makes our lives easier.

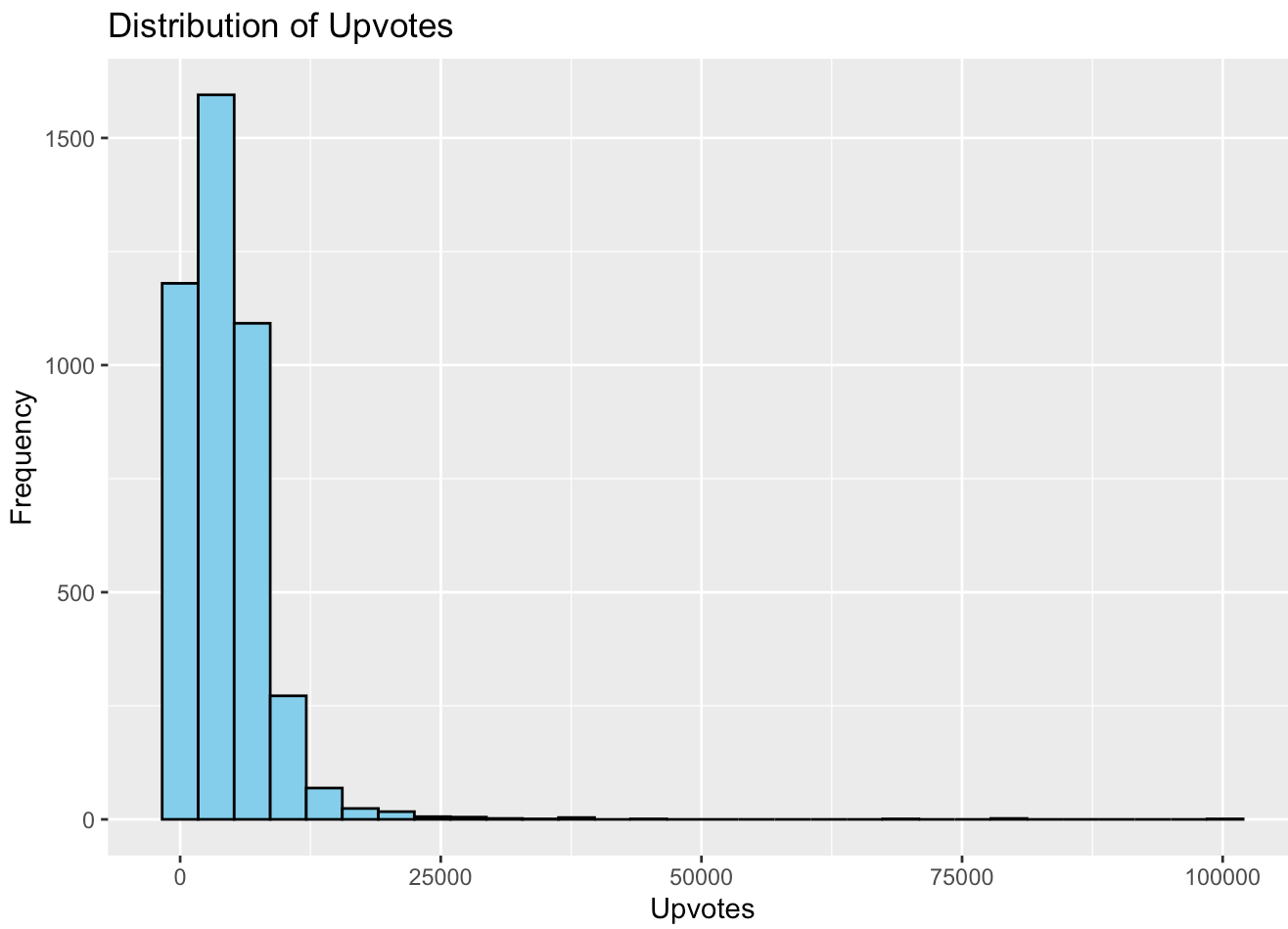
Distribution of Upvotes

Now, to get a feel for our data - and what should qualify as succesful, let's visualize the distribution of upvotes.

Hide

```
# histogram of upvotes
ggplot(urban_dictionary_data, aes(x = up)) +
  geom_histogram(fill = "skyblue", color = "black") +
  labs(title = "Distribution of Upvotes",
       x = "Upvotes",
       y = "Frequency")
```

```
## `stat_bin()` using `bins = 30`. Pick better value with `binwidth`.
```



Upvotes Summary Statistics

Just to get a more comprehensive look at the distribution of upvotes – let's look at the summary statistics.

Hide

```
summary(urban_dictionary_data$up)
```

```
##    Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
##       3    1528    3772    4414    6164  100236
```

We have quite a large range of upvotes - with the minimum number of upvotes in our dataset being 3, the maximum being 100,236, and the mean number of upvotes centered at 4,114. Out of pure curiosity - and as a sanity check, what are the most upvoted words as of the time this data scraping, and what do their indicator variables look like?

Hide

```
sorted_data <- urban_dictionary_data[order(urban_dictionary_data$up, decreasing = T
RUE), ]
top_5_most_upvoted <- sorted_data[1:5, ]
top_5_most_upvoted_columns <- top_5_most_upvoted[, c("word", "has_tags", "politica
l", "explicit")]
print(kable(top_5_most_upvoted_columns))
```

```
##
##
## |     |word      |has_tags |political |explicit |
## |:----|:---------|:--------|:---------|:--------|
## |2063 |love      |FALSE    |FALSE     |FALSE    |
## |2124 |gate rape |TRUE     |FALSE     |FALSE    |
## |2150 |gate rape |TRUE     |FALSE     |FALSE    |
## |1922 |fuck      |FALSE    |FALSE     |TRUE     |
## |2129 |nerd      |FALSE    |FALSE     |FALSE    |
```

It looks like there's a nice dispersion of TRUE/FALSE amongst our three indicator variables `has_tags`, `political`, and `explicit`. Time to move onwards!

Model Set-Up

Now that we have a better feel for our data, we can start building effective models to predict what makes a good Urban Dictioanry word! In this section, we will split our data into training and testing sets, create our recipe, and establish cross-validation.

Training/Testing Split

We begin by splitting our data into two sets, training and testing. The high level idea is that the training set will be actively used in building our models, and the testing set will be "hidden" from the models until the end of construction. By withholding the testing subset during training, we can effectively gauge our model accuracy on unseen data. For this split, I have chosen 70% of the data to go towards the training set, leaving a sufficient 30% of the data to test our model on. Additionally, to uphold model integrity, we set a seed in the beginning to ensure reproducability, and stratify our split over our outcome variable `up` , to ensure that the distribution of `up` remains consistent across both subsets.

Hide

```
set.seed(729)

# 70/30 data split
urban_dictionary_split <- initial_split(urban_dictionary_data, prop = 0.7, strata =
up)
urban_dictionary_train <- training(urban_dictionary_split)
urban_dictionary_test <- testing(urban_dictionary_split)
```

Let's double check that the split looks right:

Hide

```
percent_train <- round(nrow(urban_dictionary_train) / nrow(urban_dictionary_data) *
100, 2)
percent_test <- round(nrow(urban_dictionary_test) / nrow(urban_dictionary_data) * 1
00, 2)
paste("The training set has", percent_train, "% of the data, and the testing set ha
s", percent_test, "% of the data.")
```

```
## [1] "The training set has 69.94 % of the data, and the testing set has 30.06 % o
f the data."
```

Recipe Set-Up

Now, onto building our recipe, which serves as the heart of our predictive modeling. In the realm of machine learning, a 'recipe' is exactly what it sounds like - a crafted set of instructions that prepares our raw data for consumption by our predictive models. Visualize each variable as a distinct Urban Dictionary slang term, contributing its own flavor to the colorful tapestry of the slang linguistic world. Just as Urban Dictionary compiles a myriad of words, definitions, and hashtags into a structured encyclopedia, our recipe acts as the backbone, aiding in standardizing and organizing the raw data. By following this recipe, our models receive a clear and concise set of instructions, so they can more more effectively capture the ever-

shifting landscape of online conversation!

As mentioned above, for our predictors - or recipe ingredients if you will - we will use `has_tags`, `political`, and `explicit`. We will dummy code all our predictors since they are categorical and not continuous - a method to convert nominal data into numeric binary model terms. Since there's no missing data, we don't need to deal with imputation, and similarly, since all our predictors are categorical, we don't need to scale or center the data, just convert them to factors.

Hide

```
urban_dictionary_recipe <- recipe(up ~ has_tags + political + explicit, data = urba
n_dictionary_train) %>%
  step_mutate(has_tags = as.factor(has_tags)) %>%
  step_mutate(political = as.factor(political)) %>%
  step_mutate(explicit = as.factor(explicit)) %>%
  step_dummy(all_predictors())
```

K-Fold Cross Validation

K-fold cross-validation is a method to maximize model accuracy. With k = 10 folds in our case, the dataset is divided into 10 equal parts. The model is trained and tested 10 times, each time using a different fold for testing and the remaining folds for training. This process helps to ensure that the model's performance is not overly dependent on a single training-test split, and by averaging the results from each iteration, we obtain a better estimate of the model's performance. We stratify on the outcome, upvotes, to make sure the data in each fold is not imbalanced, and set our folds to 10.

Hide

```
urban_dictionary_folds <- vfold_cv(urban_dictionary_train, v = 10, strata = up)
```

Fitting The Models

With everything prepped and ready to go, let's fit our 5 different models: K-Nearest Neighbor, Lasso, Elastic Net, Random Forest, and Boosted Tree. Each of the 5 models roughly follows the same set-up, which will be outlined using the 5 steps below. Instead of constructing each model one-by-one, for each step, we will perform that specific step for all models simultaneously.

Perfomance Metric

One last note - To evaluate performance after model fitting, it's helpful to have a standard metric across all models to to better compare model efficiency. In our case will use Root Mean Squared Error (RMSE) since this is a regression project, and the square root of the average squared differences between predicted and observed values will provide useful insights. Whichever model is the "best" (has the lowest RMSE), will be used to test model accuracy on data that the model has never seen before (the testing set). Onwards to fitting our models!

Step 1

Set up the model by specifying the model to fit, the engine, and the mode - regression in our case.

Hide

```
# K-NEAREST NEIGHBOR
knn_model <- nearest_neighbor(neighbors = tune()) %>%
  set_mode("regression") %>%
  set_engine("kknn")

# LASSO
# Tuning penalty and setting mixture to 1 to specify lasso
lasso_model <- linear_reg(penalty = tune(),
                          mixture = 1) %>%
  set_mode("regression") %>%
  set_engine("glmnet")

# ELASTIC NET
# Tuning penalty and mixture
elastic_model <- linear_reg(penalty = tune(),
                            mixture = tune()) %>%
  set_mode("regression") %>%
  set_engine("glmnet")

# RANDOM FOREST
# Tuning mtry, trees, and min_n
rf_model <- rand_forest(mtry = tune(),
                        trees = tune(),
                        min_n = tune()) %>%
  set_engine("ranger", importance = "impurity") %>%
  set_mode("regression")

# BOOSTED TREES
# Tuning trees, learn_rate, and min_n
boosted_model<- boost_tree(trees = tune(),
                            learn_rate = tune(),
                            min_n = tune()) %>%
  set_engine("xgboost") %>%
  set_mode("regression")
```

Step 2

Set up workflow for each model by adding the new model and our recipe

Hide

```
# K-NEAREST NEIGHBOR
knn_workflow <- workflow() %>%
  add_model(knn_model) %>%
  add_recipe(urban_dictionary_recipe)

# LASSO
lasso_workflow <- workflow() %>%
  add_model(lasso_model) %>%
  add_recipe(urban_dictionary_recipe)

# ELASTIC NET
elastic_workflow <- workflow() %>%
  add_model(elastic_model) %>%
  add_recipe(urban_dictionary_recipe)

# RANDOM FOREST
rf_workflow <- workflow() %>%
  add_model(rf_model) %>%
  add_recipe(urban_dictionary_recipe)

# BOOSTED TREES
boosted_workflow <- workflow() %>%
  add_model(boosted_model) %>%
  add_recipe(urban_dictionary_recipe)
```

Step 3

Set up tuning grid to specify the ranges of the parameters you wish to tune as well as how many levels of tuning we want for each parameter

Hide

```
# K-NEAREST NEIGHBOR
knn_grid <- grid_regular(neighbors(range = c(1,10)), levels = 5)

# LASSO REGRESSION
penalty_grid <- grid_regular(penalty(range = c(-5,5)), levels = 50)

# ELASTIC NET
elastic_grid <- grid_regular(penalty(range = c(-5, 5)), mixture(range = c(0,1)), le
vels = 10)

# RANDOM FOREST
rf_grid <- grid_regular(mtry(range = c(1, 3)), trees(range = c(200,1000)), min_n(ra
nge = c(5,20)), levels = 8)

# BOOSTED TREES
boosted_grid <- grid_regular(trees(range = c(5, 200)), learn_rate(range = c(0.01,0.
1), trans = identity_trans()), min_n(range = c(40, 60)), levels = 5)
```

Step 4

Tune the model by specifying the workflow, k-fold cross validation folds, and the tuning grid

Hide

```
# K-NEAREST NEIGHBOR
knn_tune <- tune_grid(
    knn_workflow,
    resamples = urban_dictionary_folds,
    grid = knn_grid)

# LASSO
lasso_tune <- tune_grid(
  lasso_workflow,
  resamples = urban_dictionary_folds,
  grid = penalty_grid
)
```

```
## → A | warning: A correlation computation is required, but `estimate` is constant
and has 0
##                standard deviation, resulting in a divide by 0 error. `NA` will b
e returned.
```

```
##
There were issues with some computations   A: x1

There were issues with some computations   A: x2

There were issues with some computations   A: x3

There were issues with some computations   A: x4

There were issues with some computations   A: x5

There were issues with some computations   A: x6

There were issues with some computations   A: x7

There were issues with some computations   A: x8

There were issues with some computations   A: x9

There were issues with some computations   A: x10

There were issues with some computations   A: x10
```

Hide

```
# ELASTIC NET
elastic_tune <- tune_grid(
  elastic_workflow,
  resamples = urban_dictionary_folds,
  grid = elastic_grid
)
```

```
## → A | warning: A correlation computation is required, but `estimate` is constant
and has 0
##                standard deviation, resulting in a divide by 0 error. `NA` will b
e returned.
##
There were issues with some computations   A: x1

There were issues with some computations   A: x2

There were issues with some computations   A: x3

There were issues with some computations   A: x4

There were issues with some computations   A: x5

There were issues with some computations   A: x6

There were issues with some computations   A: x7

There were issues with some computations   A: x8

There were issues with some computations   A: x9

There were issues with some computations   A: x10

There were issues with some computations   A: x10
```

Hide

```
# RANDOM FOREST
rf_tune_res <- tune_grid(
  rf_workflow,
  resamples = urban_dictionary_folds,
  grid = rf_grid
)

# BOOSTED TREES
boosted_tune_res <- tune_grid(
  boosted_workflow,
  resamples = urban_dictionary_folds,
  grid = boosted_grid
)
```

Step 5

Collect the metrics of the tuned models, specifically the RMSE metric

Hide

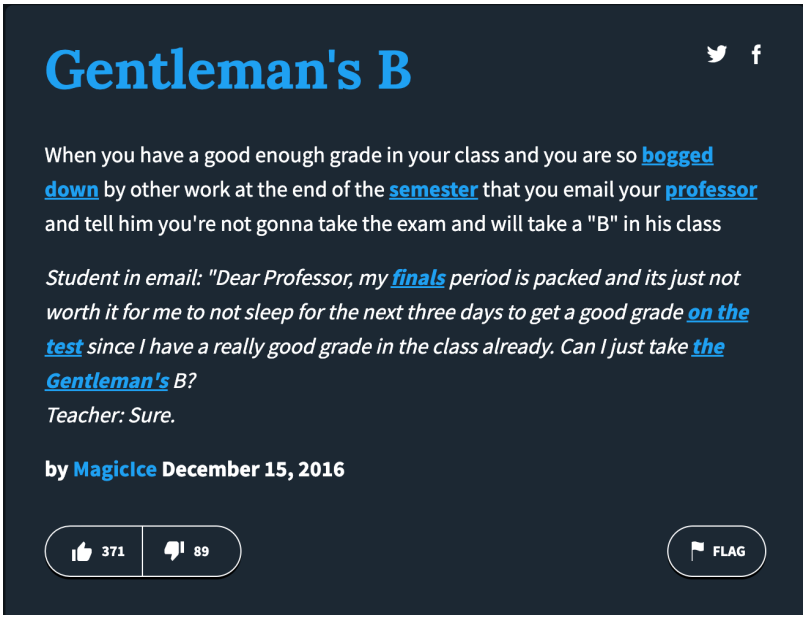
```
# K-NEAREST NEIGHBOR
knn_rmse <- collect_metrics(knn_tune) %>%
  filter(.metric == "rmse") %>%
  summarise(mean_rmse = mean(mean)) %>%
  pull(mean_rmse)

# LASSO
lasso_rmse <- collect_metrics(lasso_tune) %>%
  filter(.metric == "rmse") %>%
  summarise(mean_rmse = mean(mean)) %>%
  pull(mean_rmse)

# ELASTIC NET
elastic_rmse <- collect_metrics(elastic_tune) %>%
  filter(.metric == "rmse") %>%
  summarise(mean_rmse = mean(mean)) %>%
  pull(mean_rmse)

# RANDOM FOREST
rf_rmse <- collect_metrics(rf_tune_res) %>%
  filter(.metric == "rmse") %>%
  summarise(mean_rmse = mean(mean)) %>%
  pull(mean_rmse)

# BOOSTED TREES
boosted_rmse <- collect_metrics(boosted_tune_res) %>%
  filter(.metric == "rmse") %>%
  summarise(mean_rmse = mean(mean)) %>%
  pull(mean_rmse)
```



Model Results

Results Table

Hide

```
model_summary <- c(
  K_Nearest_Neighbors = knn_rmse,
  Lasso_Regression = lasso_rmse,
  Elastic_Net = elastic_rmse,
  Random_Forest = rf_rmse,
  Boosted_Trees = boosted_rmse
)

print(kable(model_summary))
```

```
##
##
## |                   |        x|
## |:------------------|--------:|
## |K_Nearest_Neighbors | 5714.692|
## |Lasso_Regression    | 4615.481|
## |Elastic_Net         | 4614.729|
## |Random_Forest       | 4610.621|
## |Boosted_Trees       | 4893.188|
```

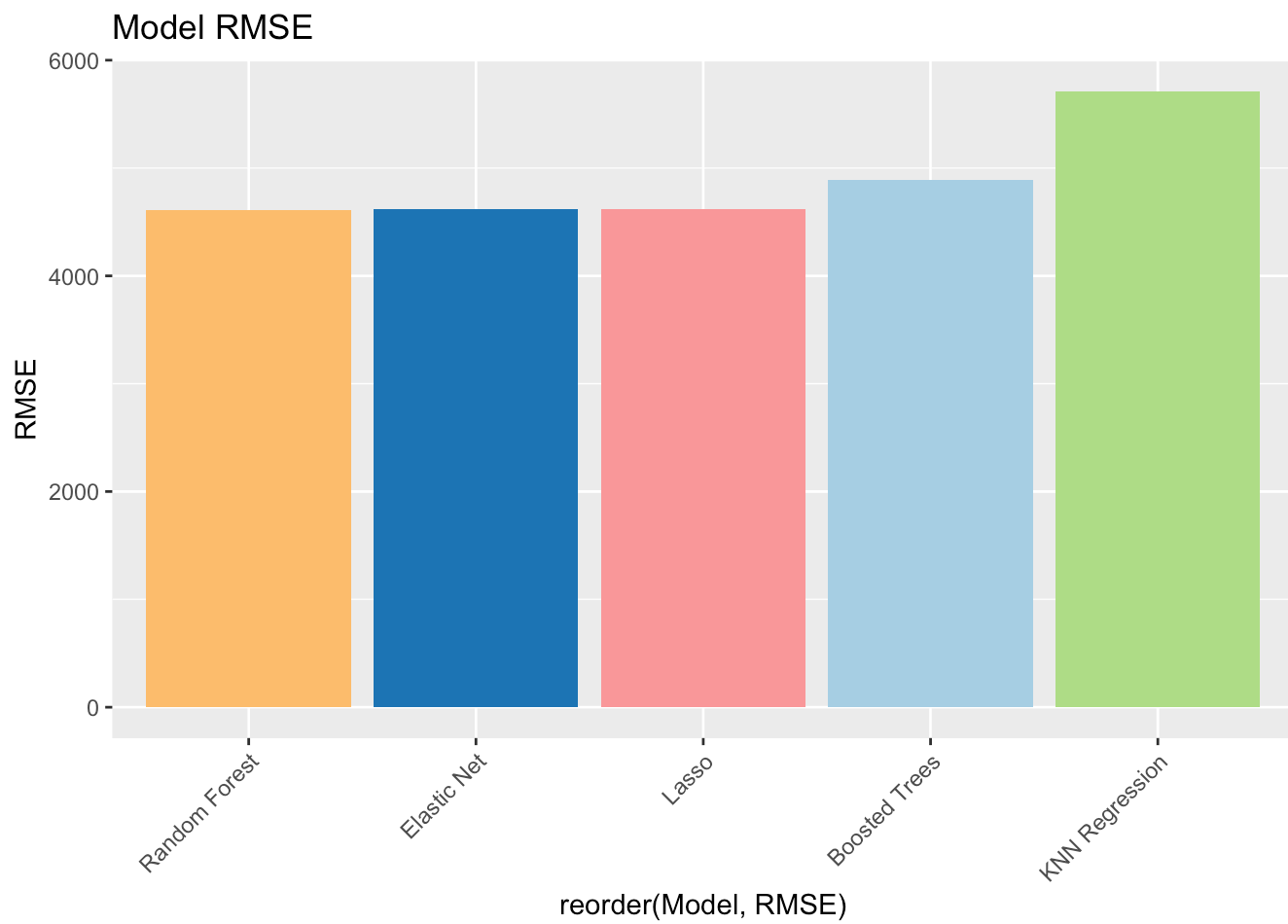
Results Graph

Hide

```
all_models <- data.frame(Model = c("KNN Regression", "Lasso", "Elastic Net", "Rando
m Forest", "Boosted Trees"),
                         RMSE = c(knn_rmse, lasso_rmse, elastic_rmse, rf_rmse, boos
ted_rmse))

palette <- c("#a6cee3", "#1f78b4", "#b2df8a", "#fb9a99", "#fdbf6f")

# Creating a barplot of the RMSE values
ggplot(all_models, aes(x = reorder(Model, RMSE), y = RMSE, fill = Model)) +
  geom_bar(stat = "identity", aes(fill = Model)) +
  scale_fill_manual(values = palette) +
  theme(legend.position = "none", axis.text.x = element_text(angle = 45, hjust =
1)) +
  labs(title = "Model RMSE")
```



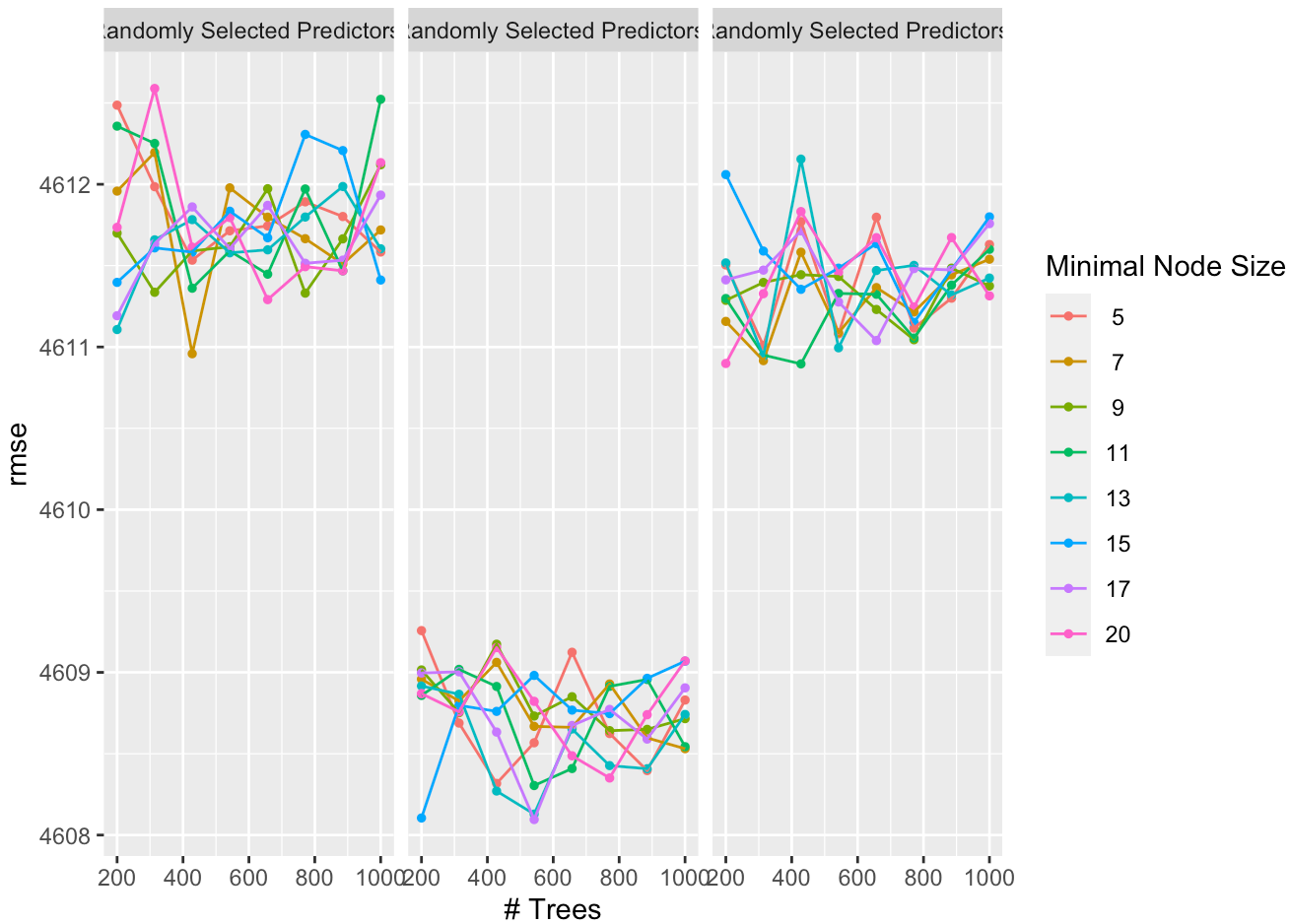
From the chart and table, we can see the Random Forest performed the best, but for the sake of exploration, we'll take our top 2 strongest models, Random Forest and Elastic Net to do some further exploration.

Autoplot

Autoplots are a handy tool to visualize the results of our tuned models. Specifically, visualizing the effects of parameters on our metric of interest, RMSE. Again, a lower RMSE indicates a better performance.

Hide

```
autoplot(rf_tune_res, metric = 'rmse')
```

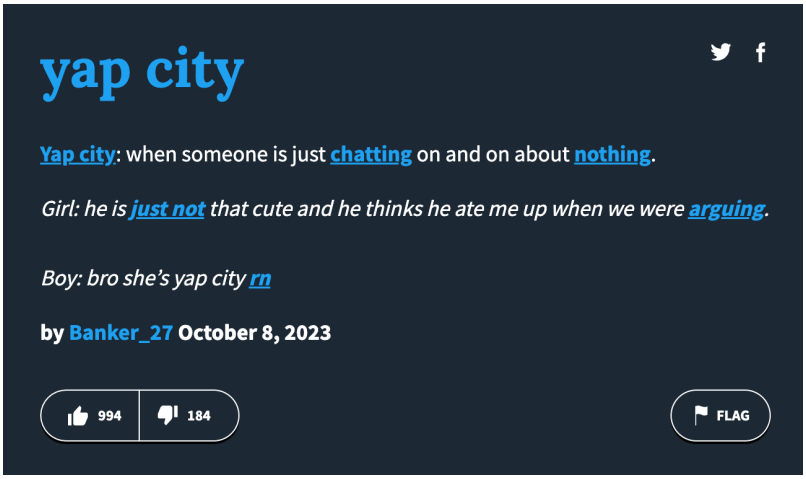


Hide

```
best_rf <- select_best(rf_tune_res, "rmse")
best_rf
```

```
## # A tibble: 1 × 4
##    mtry trees min_n .config
##   <int> <int> <int> <chr>
## 1     2   542    17 Preprocessor1_Model155
```

The hyperparemter `mtry` represents the amount of predictors randomly sampled at each split of the descison tree, `trees` represents the number of trees, and `min_n` represents the minimum number of observations needed to split at a point in the tree. The best perfrming modefl had a `mtry` value of 2, `trees` vale of 542, and `min_n` value of 17.



Putting the Best Models to Test

Performance on Folds

So, the random forest performed the best out of all 5 of our models. But which tuned parameters values were the best random forest model?

Hide

```
best_rf_model <- show_best(rf_tune_res, metric = "rmse")
best_rf_model <- best_rf_model[which.min(best_rf_model$mean), ]

best_rf_model
```

```
## # A tibble: 1 × 9
##    mtry trees min_n .metric .estimator  mean     n std_err .config
##   <int> <int> <int> <chr>   <chr>      <dbl> <int>   <dbl> <chr>
## 1     2   542    17 rmse    standard   4608.    10    433. Preprocessor1_Model1…
```

Our lucky winner is Random Forest #155 with 2 predictors, 542 trees, and a minimal node size of 17, and an RMSE of 4608.096!

Fitting to Training Data

Now, we will take that best model from the tuned random forest and fit it to the training data, which will train the model on the entire training data set. Once we have fit to the entire training data, it will be ready for our test set!

Hide

```
# Fit best performing model to whole trianing set
best_rf_train <- select_best(rf_tune_res, metric = 'rmse')
rf_final_workflow_train <- finalize_workflow(rf_workflow, best_rf_train)
rf_final_fit <- fit(rf_final_workflow_train, data = urban_dictionary_train)
```

Testing the Model

Hide

```
# Creating the predicted vs. actual value tibble
urban_dictionary_tibble <- predict(rf_final_fit, new_data = urban_dictionary_test %
>% select(-up))
urban_dictionary_tibble <- bind_cols(urban_dictionary_tibble, urban_dictionary_test
%>% select(up))
urban_dictionary_metric <- metric_set(rmse)

# Collecting the rmse of the model on the testing data
urban_dictionary_tibble_metrics <- urban_dictionary_metric(urban_dictionary_tibble,
truth = up, estimate = .pred)
urban_dictionary_tibble_metrics
```

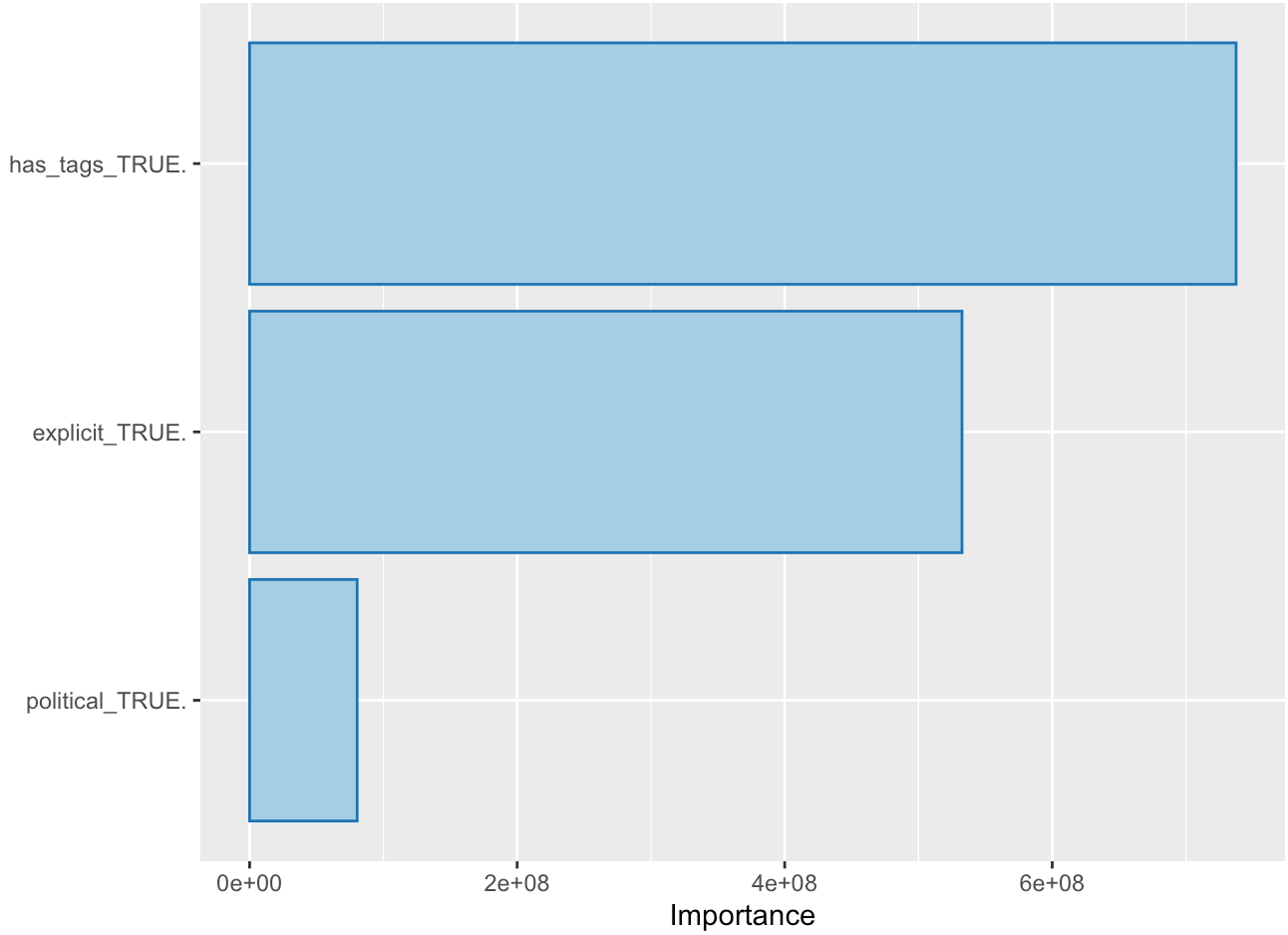
```
## # A tibble: 1 × 3
##   .metric .estimator .estimate
##   <chr>   <chr>          <dbl>
## 1 rmse    standard       3627.
```

The model performed better on our testing set, yielding a lower RMSE value!

Variable Importance Chart

Hide

```
rf_final_fit %>%
  extract_fit_engine() %>%
  vip(aesthetics = list(fill = "#a6cee3", color = "#1f78b4"))
```



From this plot, we can see whether a word was posted with tags was the most important predictor in forecasting upvotes, followed by whether the word contained explicit matter, and largely trailed by whether the word contained political matter.

Conclusion

After fitting our 5 models, we found that The Random Forest Model yielded the most accuracy on predicting the upvotes of an URban Dictionary word, though admittedly still a poor-performing model. It's not surprising though that The Random Forest Model came out on top as it's nonparemtric and more flexible. We also found that the poorest perfoming model by far was K-Nearest Neighbors.

In the process of categorizing Urban Dictionary words into political and explicit categories, it became evident that there were inherent imperfections in my system. I believe my classification system was sufficient for the sake of this machine learning exploration, but it certainly missed, and falsely included words, due to my more human sorting system. If I were to take this project further, I would look into more advanced methods to label the words into boxes, particularly natural language processing (NLP) techniques, which would allow for a more automated system for labeling Urban Dictionary words. Additionally, I could explore other - additional specific - categories to predict virality of a word, beyond political and explicit.

Overall, even though the model wasn't perfect or the most accurate, I feel a huge sense of accomplishment in my growth of machine learning knowledge. These skills and techniques of model building that I worked on during this project will be so useful as I take on more ML projects. I'm very grateful for all this class has taught me, and the fun conversation starter I now have about the virality of Urban Dictionary words, and the relevant topic of predicting virality in general.

the end

Two words that can bring a pleasing feeling at the end of a movie or send shivers down your back when talking about your life.

The end is the closest thing you can see in all of your nightmares and every haunting tale, but the comfort of knowing that green eggs and ham are a respectable dish when the book is done.

The End.

by jaylabs March 8, 2003

57 22 FLAG